Dedeepya Ramidi

WEEK – 1 :

Module 1 - Design Patterns and Principles

Software design principles are a set of best practices that guide developers in creating maintainable and extensible software systems. Following design principles helps organizations reduce technical debt, improve software quality, and simplify future modifications.

The primary objectives of software design principles are:

• Improve code readability and maintainability.

• Reduce dependency between software modules.

• Increase flexibility and scalability of applications.

• Simplify software testing and debugging.

• Promote code reuse and modularity.

• Enable easier adaptation to changing business requirements.

In enterprise applications, multiple developers often work on the same codebase. Without proper design principles, modifications in one module may unintentionally affect other modules, resulting in tightly coupled systems. SOLID principles help solve this problem by promoting separation of responsibilities, abstraction, loose coupling, and extensibility.

The five SOLID principles are:

1. Single Responsibility Principle (SRP)
2. Open Closed Principle (OCP)
3. Liskov Substitution Principle (LSP)
4. Interface Segregation Principle (ISP)
5. Dependency Inversion Principle (DIP)

Together, these principles create a strong foundation for developing reliable object-oriented applications.

Single Responsibility Principle (SRP)

Definition:
A class should have only one responsibility and one reason to change. Each class should focus on a single task or functionality. This makes the code easier to maintain, test, and understand.

Example:
In a Library Management System, a `Book` class should manage book details only, while a separate `ReportGenerator` class should generate reports.

SRP.java > ...

```java
class Student1 {
    private String name;
    Student1(String name) {
        this.name = name;
    }
    String getName1() {
        return name;
    }
}
class ReportCard {
    void printReport(Student1 student) {
        System.out.println("Student Name: " + student.getName1());
    }
}
public class SRP {
    Run | Debug
    public static void main(String[] args) {
        Student1 s = new Student1(name: "Dedeepya");
        ReportCard report = new ReportCard();
        report.printReport(s);
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling>  & 'C:\Pro
 ' 'C:\Users\Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorage\84e7d13cab

Student Name: Dedeepya
```

Open-Closed Principle (OCP)

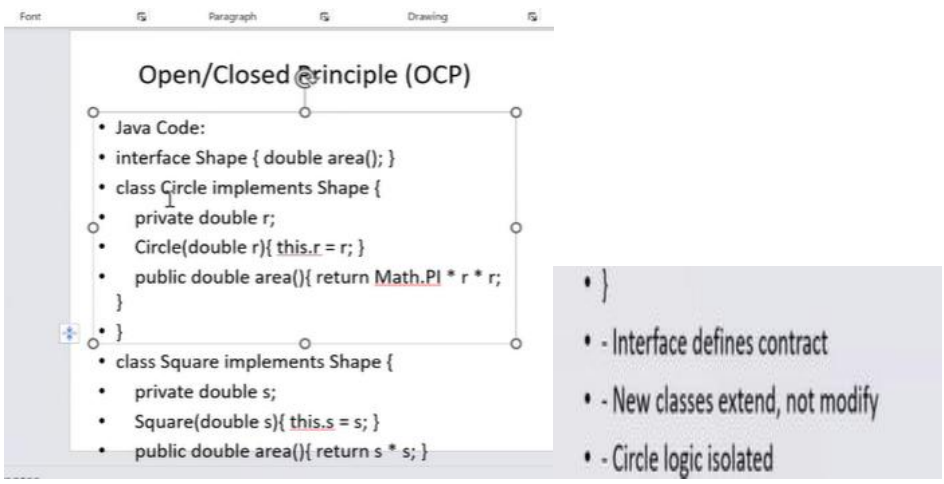
Definition:
Software entities should be open for extension but closed for modification. New features should be added by extending existing code rather than changing already tested code.

Example:
In a Shape application, adding a `Triangle` class should not require modifying existing `Circle` or `Rectangle` classes.

Dedeepya Ramidi

```java
interface Shape {
    double area();
}
class Circle implements Shape {
    private double radius;
    Circle(double radius) {
        this.radius = radius;
    }
    public double area() {
        return Math.PI * radius * radius;
    }
}
class Rectangle implements Shape {
    private double length;
    private double breadth;
    Rectangle(double length, double breadth) {
        this.length = length;
        this.breadth = breadth;
    }
    public double area() {
        return length * breadth;
    }
}
public class OCPDemo {
    Run | Debug
    public static void main(String[] args) {
        Shape circle = new Circle(radius: 5);
        Shape rectangle = new Rectangle(length: 4,breadth: 6);
        System.out.println("Circle Area: " + circle.area());
        System.out.println("Rectangle Area: " + rectangle.area());
    }
}
```

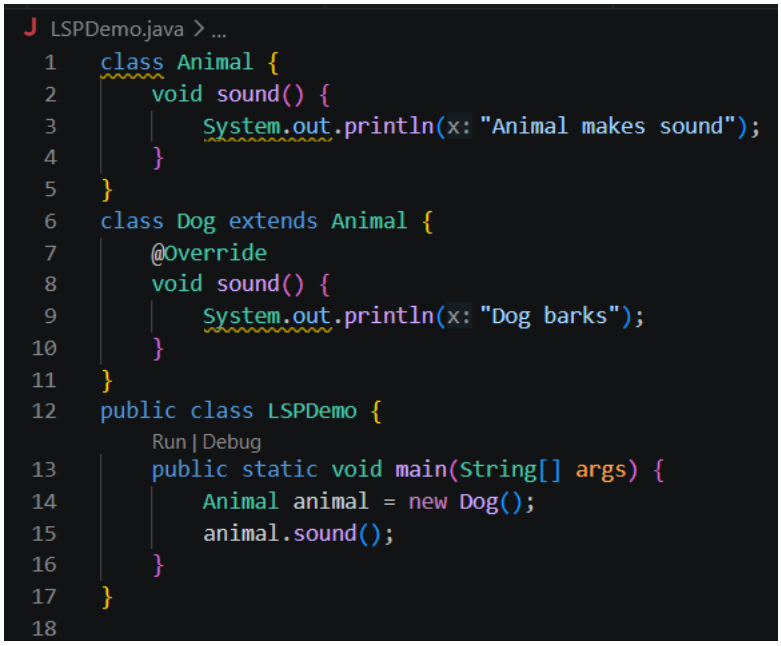
```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling
' 'C:\Users\Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorag
emo'
Circle Area: 78.53981633974483
Rectangle Area: 24.0
```

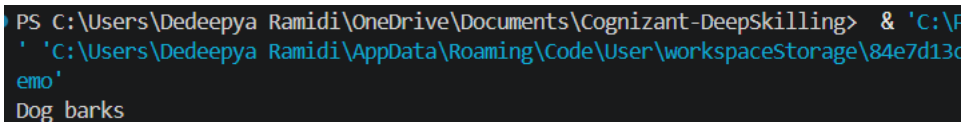


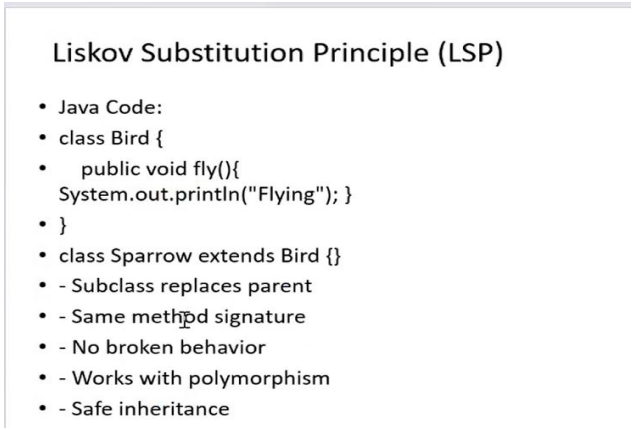
Liskov Substitution Principle (LSP)

Definition:
Objects of a subclass should be replaceable with objects of the parent class without affecting program correctness.A child class should behave in a way that does not break the expectations established by the parent class.

Example:
If `Dog` extends `Animal`, then a `Dog` object should work correctly wherever an `Animal` object is expected.





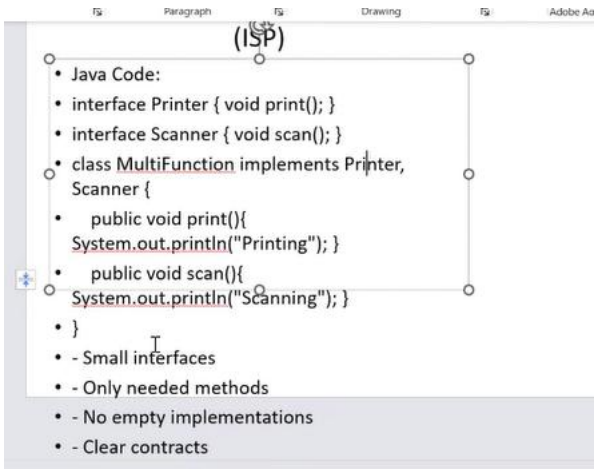


Interface Segregation Principle (ISP)

Definition:
Clients should not be forced to depend on methods they do not use.Large interfaces should be split into smaller, more specific interfaces so that classes implement only the methods they need.

Example:
Instead of creating one interface with `print()`, `scan()`, and `fax()`, separate them into `Printer`, `Scanner`, and `Fax` interfaces.

```java
// ISPDemo.java > ...
interface Printer {
    void print();
}
interface Scanner {
    void scan();
}
class MultiFunctionMachine implements Printer, Scanner {
    public void print() {
        System.out.println(x: "Printing...");
    }
    public void scan() {
        System.out.println(x: "Scanning...");
    }
}
public class ISPDemo {
    Run | Debug
    public static void main(String[] args) {
        MultiFunctionMachine machine =new MultiFunctionMachine();
        machine.print();
        machine.scan();
    }
}
```



Dependency Inversion Principle (DIP)

Definition:
High-level modules should depend on abstractions rather than concrete implementations. Classes should interact through interfaces or abstract classes, reducing dependency and increasing flexibility.

Example:
A notification system should depend on a `MessageService` interface instead of directly depending on `EmailService` or `SMSService` classes.

```java
interface MessageService {
    void sendMessage(String message);
}
class EmailService implements MessageService {
    public void sendMessage(String message) {
        System.out.println("Email: " + message);
    }
}
class Notification {
    private MessageService service;
    Notification(MessageService service) {
        this.service = service;
    }
    void notifyUser(String message) {
        service.sendMessage(message);
    }
}
public class DIPDemo {
    Run | Debug
    public static void main(String[] args) {
        MessageService service =new EmailService();
        Notification notification =new Notification(service);
        notification.notifyUser(message: "Welcome to Cognizant!");
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling>  & 'C:\Program F
' 'C:\Users\Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorage\84e7d13cabfe19d8
emo'
Email: Welcome to Cognizant!
```

```java
import com.LibraryManagementSystem.entity.User;

public interface UserService {
    @Transactional
    User addUser(UserRequestDTO user);
    @Transactional(readOnly = true)
    User getUserById(Long userId);

    List<UserResponseDTO> getUsersByRole(Roles role);

    List<UserResponseDTO> getAllMembers();

    Optional<UserResponseDTO> updateMember(int id, UserRequestDTO update

    void deleteMember(Long id);


    List<UserResponseDTO> searchByName(String name);

    List<UserResponseDTO> searchByEmail(String email);
}
```

Singleton Pattern

Definition:
Singleton Pattern ensures that a class has only one instance and provides a global point of access to that instance.It restricts object creation so that only one object exists throughout the application. This helps save memory and maintain consistency.

Example : A database connection manager where only one connection object is shared across the application.

```java
class Singleton {
    private Singleton() {}
    public static Singleton getInstance() {
        if(instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
    public void showMessage() {
        System.out.println(x: "Singleton Object Created");
    }
}
public class SingletonDemo {
    Run | Debug
    public static void main(String[] args) {
        Singleton s1 = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();
        s1.showMessage();
        System.out.println(s1 == s2);
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling>  & 'C:\Progr
wCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Dedeepya Ramidi\AppData\Roaming\Co
88d4707d2008f\redhat.java\jdt_ws\Cognizant-DeepSkilling_bf120b6\bin' 'SingletonDemo
Singleton Object Created
true
```

Builder Pattern

Definition:
Builder Pattern constructs complex objects step by step.It separates the construction process from the final object, making it easier to create objects with different configurations.

Example:
Building a computer by choosing different components such as CPU, RAM, and Storage step by step.

```java
class Student {
    private String name;
    private int age;
    private Student(Builder builder) {
        this.name = builder.name;
        this.age = builder.age;
    }
    void display() {
        System.out.println(name + " " + age);
    }
    static class Builder {
        private String name;
        private int age;
        Builder setName(String name) {
            this.name = name;
            return this;
        }
        Builder setAge(int age) {
            this.age = age;
            return this;
        }
        Student build() {
            return new Student(this);
        }
    }
}
public class BuilderDemo {
    Run | Debug
    public static void main(String[] args) {
        Student s = new Student.Builder()
                .setName(name: "Ram")
                .setAge(age: 20)
                .build();
        s.display();
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling>  & 'C:\Pro
owCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Dedeepya Ramidi\AppData\Roaming
9088d4707d2008f\redhat.java\jdt_ws\Cognizant-DeepSkilling_bf120b6\bin' 'BuilderDe
Ram 20
```

Factory Pattern

Definition:
Factory Pattern provides an interface for creating objects without exposing the object creation logic. The factory decides which object should be created based on user input or requirements.

Example:
A Vehicle Factory that creates Car, Bike, or Truck objects based on the selected vehicle type.

```
FactoryDemo.java > ...
1   interface Vehicle {
2       void drive();
3   }
4   class Car implements Vehicle {
5       public void drive() {
6           System.out.println(x: "Driving Car");
7       }
8   }
9   class Bike implements Vehicle {
10      public void drive() {
11          System.out.println(x: "Driving Bike");
12      }
13  }
14  class VehicleFactory {
15      public static Vehicle getVehicle(String type) {
16          if(type.equalsIgnoreCase(anotherString: "car"))
17              return new Car();
18          return new Bike();
19      }
20  }
21  public class FactoryDemo {
        Run | Debug
22      public static void main(String[] args) {
23          Vehicle v = VehicleFactory.getVehicle(type: "car");
24          v.drive();
25      }
26  }
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling>  & 'C:\
les\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp'
Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorage\84e7d13cabfe19d8190
08f\redhat.java\jdt_ws\Cognizant-DeepSkilling_bf120b6\bin' 'FactoryDemo'
Driving Car
```

Adapter Pattern

Definition:
Adapter Pattern allows incompatible interfaces to work together.

Explanation:
It acts as a bridge between two classes by converting one interface into another expected by the client.

Example:
A mobile charger adapter that allows a Type-C charger to connect to a device with a different charging port.

```java
interface Charger {
    void charge();
}
class OldCharger {
    void oldCharge() {
        System.out.println(x: "Old Charger");
    }
}
class Adapter implements Charger {
    private OldCharger charger;
    Adapter(OldCharger charger) {
        this.charger = charger;
    }
    public void charge() {
        charger.oldCharge();
    }
}
public class AdapterDemo {
    Run | Debug
    public static void main(String[] args) {
        Charger charger = new Adapter(new OldCharger());
        charger.charge();
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling>
les\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages'
Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorage\84e7d13cabfe1
08f\redhat.java\jdt_ws\Cognizant-DeepSkilling_bf120b6\bin' 'AdapterDemo'
Old Charger
```

Decorator Pattern

Definition:
Decorator Pattern adds new functionality to an object dynamically without modifying its original structure. Additional features can be attached to an object at runtime using wrapper classes.

Example:
Adding milk, chocolate, or cream to a coffee without changing the original coffee class.

```java
interface Coffee {
    String getDescription();
}
class BasicCoffee implements Coffee {
    public String getDescription() {
        return "Coffee";
    }
}
class MilkDecorator implements Coffee {
    private Coffee coffee;
    MilkDecorator(Coffee coffee) {
        this.coffee = coffee;
    }
    public String getDescription() {
        return coffee.getDescription() + " + Milk";
    }
}
public class DecoratorDemo {
    public static void main(String[] args) {
        Coffee coffee = new MilkDecorator(new BasicCoffee());
        System.out.println(coffee.getDescription());
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant
DeepSkilling>  & 'C:\Program Files\Java\jdk-24\bin\java.
xe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\
sers\Dedeepya Ramidi\AppData\Roaming\Code\User\workspace
torage\84e7d13cabfe19d819088d4707d2008f\redhat.java\jdt_
s\Cognizant-DeepSkilling_bf120b6\bin' 'DecoratorDemo'
Coffee + Milk
```

Observer Pattern

Definition:
Observer Pattern establishes a one-to-many relationship between objects.When the state of one object changes, all registered dependent objects are automatically notified and updated.

Example:
YouTube subscribers receiving notifications whenever a channel uploads a new video.

```java
import java.util.*;
interface Observer {
    void update(String msg);
}
class Subscriber implements Observer {
    private String name;
    Subscriber(String name) {
        this.name = name;
    }
    public void update(String msg) {
        System.out.println(name + " : " + msg);
    }
}
class Channel {
    List<Observer> list = new ArrayList<>();
    void subscribe(Observer o) {
        list.add(o);
    }
    void notifyUsers(String msg) {
        for(Observer o : list)
            o.update(msg);
    }
}
public class ObserverDemo {
    Run | Debug
    public static void main(String[] args) {
        Channel c = new Channel();
        c.subscribe(new Subscriber(name: "Alice"));
        c.subscribe(new Subscriber(name: "Bob"));
        c.notifyUsers(msg: "New Video Uploaded");
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling>  &
les\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-
Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorage\84e7d13cabfe19c
08f\redhat.java\jdt_ws\Cognizant-DeepSkilling_bf120b6\bin' 'ObserverDemo'
Alice : New Video Uploaded
Bob : New Video Uploaded
```

Strategy Pattern

Definition:
Strategy Pattern defines a family of algorithms and makes them interchangeable.Different algorithms can be selected and used at runtime without changing the client code.

Example:
A navigation app choosing different routes for Car, Bike, or Walking modes.

```java
interface PaymentStrategy {
    void pay(int amount);
}
class UPI implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid via UPI : " + amount);
    }
}
class Card implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Paid via Card : " + amount);
    }
}
class Payment {
    private PaymentStrategy strategy;
    Payment(PaymentStrategy strategy) {
        this.strategy = strategy;
    }
    void payAmount(int amount) {
        strategy.pay(amount);
    }
}
public class StrategyDemo {
    public static void main(String[] args) {
       Payment p = new Payment(new UPI());
        p.payAmount(amount: 1000);
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling>
les\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages'
Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorage\84e7d13cabfe
08f\redhat.java\jdt_ws\Cognizant-DeepSkilling_bf120b6\bin' 'StrategyDem
Paid via UPI : 1000
```

Facade Pattern

Definition:
Facade Pattern provides a simplified interface to a complex subsystem.It hides the complexity of multiple classes and exposes a single interface for the client.

Example:
An online banking application where one button performs multiple operations such as authentication, balance verification, and transaction processing.

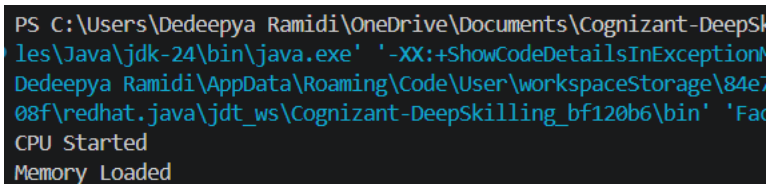
MVC (Model-View-Controller) Pattern

Definition:
MVC is an architectural pattern that separates an application into Model, View, and

Controller components.The Model manages data, the View displays information, and the Controller handles user input and communication between the Model and View.

Example:
In a banking application, account information is stored in the Model, displayed through the View, and managed by the Controller.

```
FacadeDemo.java > FacadeDemo > main(String[])
1   class CPU {
2       void start() {
3           System.out.println(x: "CPU Started");
4       }
5   }
6   class Memory {
7       void load() {
8           System.out.println(x: "Memory Loaded");
9       }
10  }
11  class ComputerFacade {
12      CPU cpu = new CPU();
13      Memory memory = new Memory();
14      void startComputer() {
15          cpu.start();
16          memory.load();
17      }
18  }
19  public class FacadeDemo {
        Run | Debug
20      public static void main(String[] args) {
21          ComputerFacade computer = new ComputerFacade();
22          computer.startComputer();
23      }
24  }
```



```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSk
les\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionM
Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorage\84e7
08f\redhat.java\jdt_ws\Cognizant-DeepSkilling_bf120b6\bin' 'Fac
CPU Started
Memory Loaded
```

MVVM (Model-View-ViewModel) Pattern

Definition:
MVVM is an architectural pattern that separates the user interface from business logic using a ViewModel. The ViewModel acts as a bridge between the View and the Model, making the application easier to maintain and test. **Ex:** An Android weather application where the View displays weather information, the Model provides data, and the ViewModel manages the interaction between them.

Dedeepya Ramidi

```java
class Model {

    String getData() {
        return "Weather : Sunny";
    }
}

class ViewModel {

    private Model model = new Model();

    String getWeather() {
        return model.getData();
    }
}

class View {

    void display(String data) {
        System.out.println(data);
    }
}

public class MVVMDemo {
    Run | Debug
    public static void main(String[] args) {

        ViewModel vm = new ViewModel();
        View view = new View();

        view.display(vm.getWeather());
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Document
 ' 'C:\Users\Dedeepya Ramidi\AppData\Roaming\C
Demo'
Weather : Sunny
```

Commonly used Design Patterns GoF, Creational Patterns - Singleton Pattern, Factory Method Pattern, Builder Pattern; Structural Patterns - Adapter Pattern, Decorator Pattern, Proxy Pattern; Behavioral Patterns - Observer Pattern, Strategy Pattern, Command Pattern; Architectural Patterns - Model-View Controller (MVC), Dependency Injection

Pattern	Kingdom Story	Remember
Singleton	One King	One object only
Factory	Toy Factory	Creates objects
Builder	Burger Chef	Build step-by-step
Adapter	Charger Adapter	Connect incompatible things
Decorator	Coffee Toppings	Add features
Proxy	Security Guard	Control access
Observer	Newspaper Subscription	Notify subscribers
Strategy	Travel Options	Choose behavior
Command	King's Orders	Request as object
MVC	Bank App	Separate data, UI, control
Dependency Injection	Blacksmith gives sword	Dependencies come from outside

Create Objects → Singleton, Factory, Builder

Modify Structure → Adapter, Decorator, Proxy, Facade

Manage Behavior → Observer, Strategy, Command

Organize Application → MVC, Dependency Injection

Dedeepya Ramidi

PRACTICE – PROBLEMS- OUTPUTS

1.Adapter Pattern

```java
public class AdapterPatternDemo {
    public static void main(String[] args) {
        PaymentProcessor payPalPayment = new PayPalAdapter(new PayPalGateway());
        PaymentProcessor stripePayment = new StripeAdapter(new StripeGateway());

        payPalPayment.pay(amount: 150.00);
        stripePayment.pay(amount: 230.50);
    }
}

interface PaymentProcessor {
    void pay(double amount);
}

class PayPalGateway {
    public void sendPayment(double amount) {
        System.out.println("Processing payment through PayPal: $" + amount);
    }
}

class StripeGateway {
    public void makePayment(double amount) {
        System.out.println("Processing payment through Stripe: $" + amount);
    }
}

class PayPalAdapter implements PaymentProcessor {
    private final PayPalGateway payPalGateway;

    public PayPalAdapter(PayPalGateway payPalGateway) {
        this.payPalGateway = payPalGateway;
    }

    @Override
    public void pay(double amount) {
        payPalGateway.sendPayment(amount);
    }
}

class StripeAdapter implements PaymentProcessor {
    private final StripeGateway stripeGateway;

    public StripeAdapter(StripeGateway stripeGateway) {
        this.stripeGateway = stripeGateway;
    }

    @Override
    public void pay(double amount) {
        stripeGateway.makePayment(amount);
    }
}
```

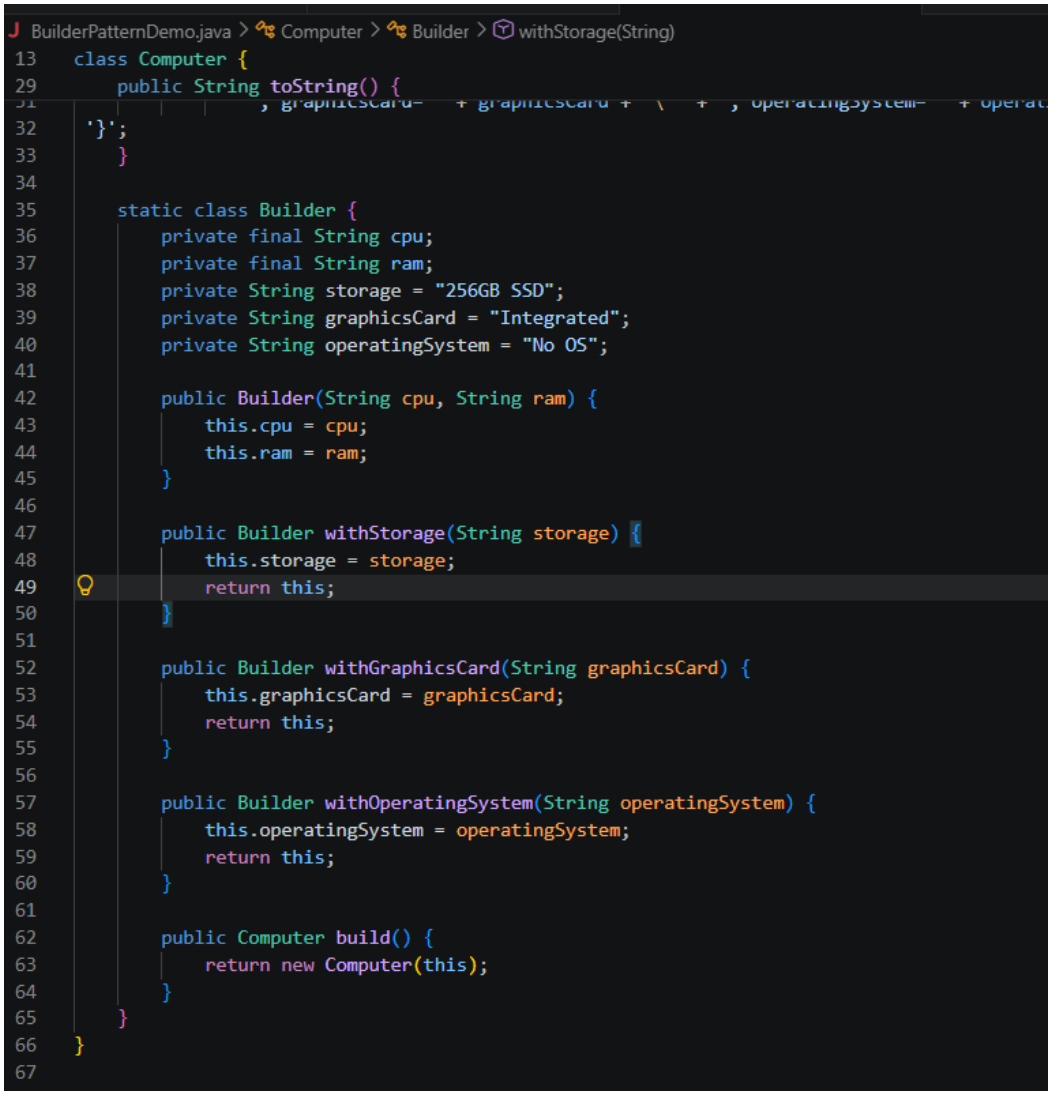
```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling\Practice - DPP>  & 'C:\Program Files\Java\jdk-24\bin\ja
8c625055a63ee1c93e6d99bc\redhat.java\jdt_ws\Practice - DPP_9c771e45\bin' 'AdapterPatternDemo'
Processing payment through PayPal: $150.0
Processing payment through Stripe: $230.5
```

2.Builder Pattern

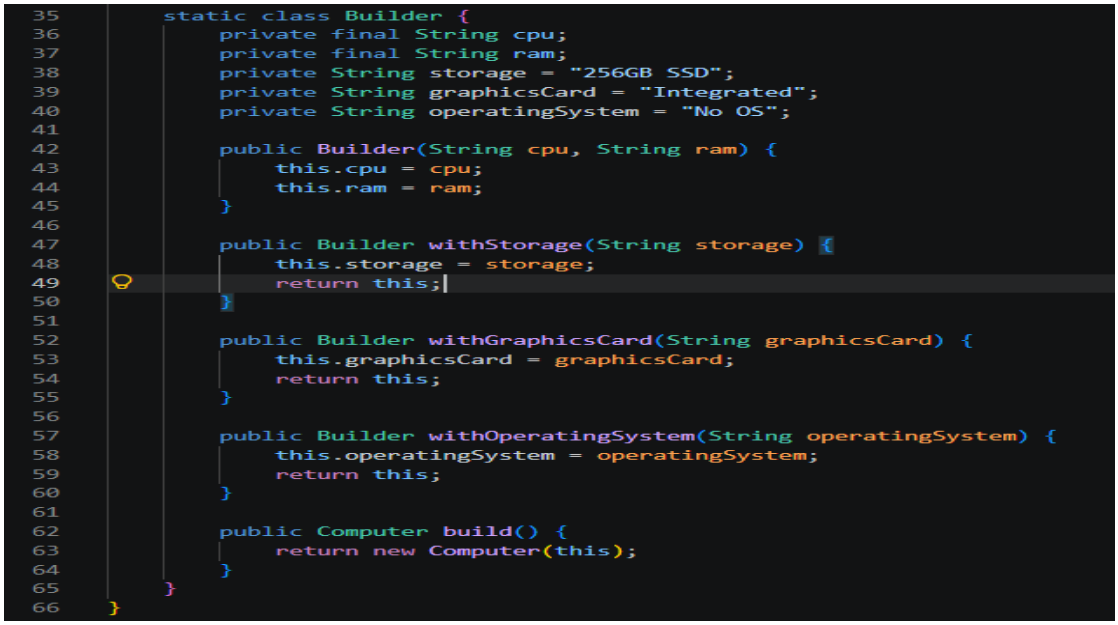
Output:

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling\Practice - DPP>  & 'C:\Program Files\Java\jdk-24\b
 ' 'C:\Users\Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorage\a2300d2d8c625055a63ee1c93e6d99bc\redhat.java\jdt_
Computer{cpu='Intel i9', ram='32GB', storage='2TB SSD', graphicsCard='NVIDIA RTX 4080', operatingSystem='Windows 11'}
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling\Practice - DPP>
```

Dedeepya Ramidi



```
BuilderPatternDemo.java > Computer > Builder > withStorage(String)
13  class Computer {
29      public String toString() {
32  '}';
33      }
34
35      static class Builder {
36          private final String cpu;
37          private final String ram;
38          private String storage = "256GB SSD";
39          private String graphicsCard = "Integrated";
40          private String operatingSystem = "No OS";
41
42          public Builder(String cpu, String ram) {
43              this.cpu = cpu;
44              this.ram = ram;
45          }
46
47          public Builder withStorage(String storage) {
48              this.storage = storage;
49              return this;
50          }
51
52          public Builder withGraphicsCard(String graphicsCard) {
53              this.graphicsCard = graphicsCard;
54              return this;
55          }
56
57          public Builder withOperatingSystem(String operatingSystem) {
58              this.operatingSystem = operatingSystem;
59              return this;
60          }
61
62          public Computer build() {
63              return new Computer(this);
64          }
65      }
66  }
67
```



```
35      static class Builder {
36          private final String cpu;
37          private final String ram;
38          private String storage = "256GB SSD";
39          private String graphicsCard = "Integrated";
40          private String operatingSystem = "No OS";
41
42          public Builder(String cpu, String ram) {
43              this.cpu = cpu;
44              this.ram = ram;
45          }
46
47          public Builder withStorage(String storage) {
48              this.storage = storage;
49              return this;
50          }
51
52          public Builder withGraphicsCard(String graphicsCard) {
53              this.graphicsCard = graphicsCard;
54              return this;
55          }
56
57          public Builder withOperatingSystem(String operatingSystem) {
58              this.operatingSystem = operatingSystem;
59              return this;
60          }
61
62          public Computer build() {
63              return new Computer(this);
64          }
65      }
66  }
```

Dedeepya Ramidi

3.Command Pattern

```java
import java.util.ArrayList;
import java.util.List;

public class CommandPatternDemo {
    Run | Debug
    public static void main(String[] args) {
        Light light = new Light();
        Command switchOn = new TurnOnCommand(light);
        Command switchOff = new TurnOffCommand(light);

        RemoteControl remote = new RemoteControl();
        remote.addCommand(switchOn);
        remote.addCommand(switchOff);
        remote.executeCommands();
    }
}

interface Command {
    void execute();
}

class Light {
    public void on() {
        System.out.println(x: "Light is ON");
    }

    public void off() {
        System.out.println(x: "Light is OFF");
    }
}

class TurnOnCommand implements Command {
    private final Light light;

    public TurnOnCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.on();
    }
}
```

```java
class TurnOffCommand implements Command {
    private final Light light;

    public TurnOffCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.off();
    }
}

class RemoteControl {
    private final List<Command> commandList = new ArrayList<>();

    public void addCommand(Command command) {
        commandList.add(command);
    }

    public void executeCommands() {
        for (Command command : commandList) {
            command.execute();
        }
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling\Practice - DPP>  &
' 'C:\Users\Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorage\a2300d2d8c625055a
Light is ON
Light is OFF
```

4.Decorator Pattern

```
class EmailNotification extends NotificationDecorator {
    public EmailNotification(Notification notification) {
        super(notification);
    }

    @Override
    public void send(String message) {
        wrappedNotification.send(message);
        sendEmail(message);
    }

    private void sendEmail(String message) {
        System.out.println("Sending email notification: " + message);
    }
}

class SMSNotification extends NotificationDecorator {
    public SMSNotification(Notification notification) {
        super(notification);
    }

    @Override
    public void send(String message) {
        wrappedNotification.send(message);
        sendSMS(message);
    }

    private void sendSMS(String message) {
        System.out.println("Sending SMS notification: " + message);
    }
}

```

DecoratorPatternDemo.java > Notification

```
public class DecoratorPatternDemo {
    Run | Debug
    public static void main(String[] args) {
        Notification notification = new EmailNotification(new BasicNotification());
        notification = new SMSNotification(notification);

        notification.send(message: "Your order has been shipped.");
    }
}

interface Notification {
    void send(String message);
}

class BasicNotification implements Notification {
    @Override
    public void send(String message) {
        System.out.println("Notification: " + message);
    }
}

abstract class NotificationDecorator implements Notification {
    protected Notification wrappedNotification;

    public NotificationDecorator(Notification notification) {
        this.wrappedNotification = notification;
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling\Practice - DPP>  & 'C:
' 'C:\Users\Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorage\a2300d2d8c625055a63ee
Notification: Your order has been shipped.
Sending email notification: Your order has been shipped.
Sending SMS notification: Your order has been shipped.
```

Dedeepya Ramidi

5.DependencyInjection Pattern

```java
public class DependencyInjectionDemo {
    public static void main(String[] args) {
        CustomerRepository repository = new InMemoryCustomerRepository();
        CustomerService service = new CustomerService(repository);

        service.addCustomer(new Customer(id: 1, name: "John Doe"));
        service.printCustomer(id: 1);
    }
}

class Customer {
    private final int id;
    private final String name;

    public Customer(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }
}

interface CustomerRepository {
    void save(Customer customer);
    Customer findById(int id);
}
```

```java
class InMemoryCustomerRepository implements CustomerRepository {
    private final java.util.Map<Integer, Customer> data = new java.util.HashMap<>();

    @Override
    public void save(Customer customer) {
        data.put(customer.getId(), customer);
    }

    @Override
    public Customer findById(int id) {
        return data.get(id);
    }
}

class CustomerService {
    private final CustomerRepository repository;

    public CustomerService(CustomerRepository repository) {
        this.repository = repository;
    }

    public void addCustomer(Customer customer) {
        repository.save(customer);
        System.out.println("Customer added: " + customer.getName());
    }

    public void printCustomer(int id) {
        Customer customer = repository.findById(id);
        if (customer != null) {
            System.out.println("Found customer: " + customer.getName());
        } else {
            System.out.println("Customer not found with id: " + id);
        }
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling\Practice - DPP>  & 'C:\
 ' 'C:\Users\Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorage\a2300d2d8c625055a63ee1
Customer added: John Doe
Found customer: John Doe
```

6.Factory Pattern

```java
FactoryMethodPatternDemo.java > ExcelDocument
public class FactoryMethodPatternDemo {
    Run | Debug
    public static void main(String[] args) {
        Document wordDoc = DocumentFactory.createDocument(type: "WORD");
        Document pdfDoc = DocumentFactory.createDocument(type: "PDF");
        Document excelDoc = DocumentFactory.createDocument(type: "EXCEL");

        wordDoc.open();
        pdfDoc.open();
        excelDoc.open();
    }
}

interface Document {
    void open();
}

class WordDocument implements Document {
    @Override
    public void open() {
        System.out.println(x: "Opening Word document...");
    }
}

class PdfDocument implements Document {
    @Override
    public void open() {
        System.out.println(x: "Opening PDF document...");
    }
}
```

```java
class ExcelDocument implements Document {
    @Override
    public void open() {
        System.out.println(x: "Opening Excel document...");
    }
}

class DocumentFactory {
    public static Document createDocument(String type) {
        switch (type.toUpperCase()) {
            case "WORD":
                return new WordDocument();
            case "PDF":
                return new PdfDocument();
            case "EXCEL":
                return new ExcelDocument();
            default:
                throw new IllegalArgumentException("Unknown document type: " + type);
        }
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling\Practice - DPP>  & 'C:\Pr
' 'C:\Users\Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorage\a2300d2d8c625055a63ee1c9

Opening Word document...
Opening PDF document...
Opening Excel document...
```

8.MVC Pattern

MVCPatternDemo.java > MVCPatternDemo > main(String[])

```java
public class MVCPatternDemo {
    Run | Debug
    public static void main(String[] args) {
        Student model = new Student(id: 1, name: "Alice");
        StudentView view = new StudentView();
        StudentController controller = new StudentController(model, view);

        controller.updateStudentName(name: "Alice Johnson");
        controller.updateView();
    }
}
class Student {
    private int id;
    private String name;
    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
    public int getId() {
        return id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
}
class StudentView {
    public void printStudentDetails(int id, String name) {
        System.out.println(x: "Student: ");
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
    }
}
class StudentController {
    private final Student model;
    private final StudentView view;
    public StudentController(Student model, StudentView view) {
        this.model = model;
        this.view = view;
    }
    public void updateStudentName(String name) {
        model.setName(name);
    }
    public void updateView() {
        view.printStudentDetails(model.getId(), model.getName());
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-De
300d2d8c625055a63ee1c93e6d99bc\redhat.java\jdt_ws\Practice
Student:
ID: 1
Name: Alice Johnson
```

Dedeepya Ramidi

9.Observer Pattern

ObserverPatternDemo.java > ...

```java
import java.util.ArrayList;
import java.util.List;

public class ObserverPatternDemo {
    Run | Debug
    public static void main(String[] args) {
        StockMarket stockMarket = new StockMarket();
        StockObserver investorA = new StockObserver(name: "Investor A");
        StockObserver investorB = new StockObserver(name: "Investor B");

        stockMarket.addObserver(investorA);
        stockMarket.addObserver(investorB);

        stockMarket.setPrice(price: 120.0);
        stockMarket.setPrice(price: 130.5);
    }
}

interface Subject {
    void addObserver(Observer observer);
    void removeObserver(Observer observer);
    void notifyObservers();
}

interface Observer {
    void update(double price);
}
```

```java
class StockMarket implements Subject {
    private final List<Observer> observers = new ArrayList<>();
    private double price;

    public void setPrice(double price) {
        this.price = price;
        notifyObservers();
    }

    @Override
    public void addObserver(Observer observer) {
        observers.add(observer);
    }

    @Override
    public void removeObserver(Observer observer) {
        observers.remove(observer);
    }

    @Override
    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(price);
        }
    }
}

class StockObserver implements Observer {
    private final String name;

    public StockObserver(String name) {
        this.name = name;
    }

    @Override
    public void update(double price) {
        System.out.println(name + " received stock price update: " + price);
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling\Practice - DPP>
' 'C:\Users\Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorage\a2300d2d8c625055
Investor A received stock price update: 120.0
Investor B received stock price update: 120.0
Investor A received stock price update: 130.5
Investor B received stock price update: 130.5
```

10.Proxy Pattern

```java
public class ProxyPatternDemo {
    public static void main(String[] args) {
        Image image = new ImageProxy(fileName: "photo.jpg");
        image.display();
        image.display();
    }
}

interface Image {
    void display();
}

class RealImage implements Image {
    private final String fileName;

    public RealImage(String fileName) {
        this.fileName = fileName;
        loadFromRemoteServer();
    }

    @Override
    public void display() {
        System.out.println("Displaying image: " + fileName);
    }

    private void loadFromRemoteServer() {
        System.out.println("Loading image from remote server: " + fileName);
    }
}

class ImageProxy implements Image {
    private final String fileName;
    private RealImage realImage;

    public ImageProxy(String fileName) {
        this.fileName = fileName;
    }

    @Override
    public void display() {
        if (realImage == null) {
            realImage = new RealImage(fileName);
        } else {
            System.out.println("Using cached image: " + fileName);
        }
        realImage.display();
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling\Practice - DPP> & 'C:\Program Files\Java\jdk-
300d2d8c625055a63ee1c93e6d99bc\redhat.java\jdt_ws\Practice - DPP_9c771e45\bin' 'ProxyPatternDemo'
Loading image from remote server: photo.jpg
Displaying image: photo.jpg
Using cached image: photo.jpg
Displaying image: photo.jpg
```

11.Singleton Pattern

```java
public class SingletonPatternDemo {
    public static void main(String[] args) {
        LoggerSingleton logger1 = LoggerSingleton.getInstance();
        LoggerSingleton logger2 = LoggerSingleton.getInstance();

        logger1.log(message: "Starting singleton demo...");
        logger2.log(message: "Both references point to the same logger instance.");

        System.out.println("logger1 == logger2: " + (logger1 == logger2));
    }
}

class LoggerSingleton {
    private static LoggerSingleton instance;
    private LoggerSingleton() {
        // Private constructor prevents external instantiation
    }

    public static synchronized LoggerSingleton getInstance() {
        if (instance == null) {
            instance = new LoggerSingleton();
        }
        return instance;
    }

    public void log(String message) {
        System.out.println("[Logger] " + message);
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling\Practice - DPP>  &
' 'C:\Users\Dedeepya Ramidi\AppData\Roaming\Code\User\workspaceStorage\a2300d2d8c625055a6
[Logger] Starting singleton demo...
[Logger] Both references point to the same logger instance.
logger1 == logger2: true
```

12.Strategy Pattern

StrategyPatternDemo.java > StrategyPatternDemo

```java
public class StrategyPatternDemo {
    Run | Debug
    public static void main(String[] args) {
        PaymentContext context = new PaymentContext();

        context.setStrategy(new CreditCardPayment());
        context.pay(amount: 250.00);

        context.setStrategy(new PayPalPayment());
        context.pay(amount: 79.99);
    }
}

interface PaymentStrategy {
    void pay(double amount);
}

class CreditCardPayment implements PaymentStrategy {
    @Override
    public void pay(double amount) {
        System.out.println("Paying " + amount + " using Credit Card.");
    }
}

class PayPalPayment implements PaymentStrategy {
    @Override
    public void pay(double amount) {
        System.out.println("Paying " + amount + " using PayPal.");
    }
}

class PaymentContext {
    private PaymentStrategy strategy;

    public void setStrategy(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    public void pay(double amount) {
        if (strategy == null) {
            throw new IllegalStateException(s: "Payment strategy is not set.");
        }
        strategy.pay(amount);
    }
}
```

```
PS C:\Users\Dedeepya Ramidi\OneDrive\Documents\Cognizant-DeepSkilling\Pr
etailsInExceptionMessages' '-cp' 'C:\Users\Dedeepya Ramidi\AppData\Roami
dt_ws\Practice - DPP_9c771e45\bin' 'StrategyPatternDemo'
Paying 250.0 using Credit Card.
Paying 79.99 using PayPal.
```